

01 Instalación

```
dart pub add drift drift_flutter path_provider
dart pub add dev:drift_dev dev:build_runner
```

```
dependencies:
  drift: ^2.33.0
  drift_flutter: ^0.3.1 # abre la BD por plataforma
  path_provider: ^2.1.5
dev_dependencies:
  drift_dev: ^2.33.0 # generador de código
  build_runner: ^2.15.0
```

Generación de código

```
dart run build_runner build # genera una vez
dart run build_runner watch # regenera en cada cambio
```

→ Cada vez que agregas o cambias una tabla o columna hay que volver a correr el generador.

02 Clase de la base de datos

Punto de entrada central: referencia todas las tablas y genera `database.g.dart`.

```
import 'package:drift/drift.dart';
import 'package:drift_flutter/drift_flutter.dart';
import 'package:path_provider/path_provider.dart';

part 'database.g.dart';

class TodoItems extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get title =>
    text().withLength(min: 6, max: 32);
  TextColumn get content => text().named('body');
  DateTimeColumn get createdAt => dateTime().nullable();
}

@DriftDatabase(tables: [TodoItems])
class AppDatabase extends _$AppDatabase {
  AppDatabase([QueryExecutor? executor])
    : super(executor ?? _openConnection());

  @override
  int get schemaVersion => 1;

  static QueryExecutor _openConnection() {
    return driftDatabase(
      name: 'my_database',
      native: const DriftNativeOptions(
        databaseDirectory: getApplicationSupportDirectory(),
      ),
    );
  }
}
```

- `part + extends _$AppDatabase` → código generado.
- `schemaVersion` sube con cada cambio de esquema.
- `driftDatabase()` abre la BD según la plataforma; por defecto en `getApplicationDocumentsDirectory()`.

03 Tablas y columnas

- Cada tabla es una clase que extiende `Table`.
- Cada columna termina con un par extra de paréntesis: `integer().autoIncrement()`.
- Columnas **no nulas por defecto** → usa `.nullable()`.
- Nombre SQL derivado de la clase (`TodoItems` → `todo_items`); cámbialo con `tableName` / `.named()`.

Tipos de columna

DART	DRIFT	SQL
int	integer()	INTEGER
BigInt	int64()	INTEGER 64-bit
String	text()	TEXT
bool	boolean()	INTEGER 0/1
double	real()	REAL
Uint8List	blob()	BLOB
DateTime	dateTime()	INTEGER o TEXT
Enums	intEnum<T>() / textEnum<T>()	INT / TEXT
Tipo propio	type converters	según el tipo

Modificadores comunes

```
integer().autoIncrement()() // PK autoincremental
text().nullable()() // permite null
text().withLength(min: 1, max: 50)() // valida longitud
text().named('body')() // nombre en SQL
dateTime().withDefault(currentDateTime()) // DEFAULT
boolean().clientDefault(() => false)() // default en Dart
integer().references(Artists, #id)() // foreign key
integer().check(age.isBiggerOrEqualValue(0))() // CHECK
text().unique()() // UNIQUE
real().generatedAs(precio * cantidad)() // col. generada
```

Primary key personalizada y mixins

```
class Profiles extends Table {
  late final email = text();
  @override
  Set<Column<Object>> get primaryKey => {email};
}

// columnas compartidas entre tablas
mixin TableMixin on Table {
  late final id = integer().autoIncrement();
  late final createdAt =
    dateTime().withDefault(currentDateTime());
}
class Posts extends Table with TableMixin { ... }
```

⚠ En sqlite3 las foreign keys vienen **desactivadas**. Actívalas en `beforeOpen`: `await customStatement('PRAGMA foreign_keys = ON');`

04 Data class vs Companion

- Data class** (`TodoItem`): fila **completa** leída de la BD.
- Companion**: datos **parciales** para inserts/updates. Cada campo es `Value<T>`: `Value(x)` escribe (incl. `Value(null)`); `Value.absent()` no toca la columna.
- `TodoItemsCompanion.insert(...)` marca **required** las columnas obligatorias → ideal para inserts.

05 Consultas (SELECT)

```
// todos los registros
Future<List<TodoItem>> get all => select(todoItems).get();

// stream reactivo: re-emite cuando los datos cambian
Stream<List<TodoItem>> watchAll() =>
  select(todoItems).watch();

// limit + offset
(select(todoItems)..limit(10, offset: 20)).get();

// orderBy
(select(todoItems)..orderBy([(t) => OrderingTerm(
  expression: t.title, mode: OrderingMode.desc)])).get();

// where + un solo resultado
final q = select(todoItems)
  ..where((t) => t.id.equals(id));
q.getSingle(); // lanza si no hay
q.getSingleOrNull(); // null si no hay
q.watchSingle(); // stream de un registro

// map antes de get/watch
query.map((row) => row.content).watch();
```

Expresiones en `where: equals`, `isBiggerThan`, `isSmallerThanValue`, `like('%x%')`, `contains`, `isNull()` — combinables con `&`, `|`, `.not()`.

Joins

```
final query = select(todoItems).join([
  leftOuterJoin(categories,
    categories.id.equalsExp(todoItems.category)),
]);

return query.watch().map((rows) => rows.map((row) {
  return EntryWithCategory(
    row.readTable(todoItems), // lanza si falta
    row.readTableOrNull(categories), // null-safe
  );
}).toList());
```

- Operadores: `innerJoin`, `leftOuterJoin`, `rightOuterJoin`, `fullOuterJoin`, `crossJoin`.
- `alias(tabla, 'nombre')` usa la misma tabla dos veces; `useColumns: false` si el join solo filtra.
- En joins, `where` / `orderBy` reciben expresiones directas: `query.where(todos.content.like(filtro));`
- Columnas calculadas: `addColumnns([expr]) + row.read(expr)`.

08 Streams reactivos

- Cualquier select es reactivo con `.watch()` / `.watchSingle()` / `.watchSingleOrNull()`.
- Emite cuando cambian las tablas involucradas → perfecto con `StreamBuilder`.
- Streams creados **fuera** de una transacción solo se actualizan al terminar ésta.

```
StreamBuilder<List<TodoItem>>((
  stream: database.select(database.todoItems).watch(),
  builder: (context, snapshot) { ... },
));
```

06 Escrituras (INSERT · UPDATE · DELETE)

```
// INSERT — devuelve el id generado
Future<int> addTodo(TodoItemsCompanion e) =>
  into(todoItems).insert(e);

// INSERT devolviendo la fila completa
final row = await into(todoItems).insertReturning(
  TodoItemsCompanion.insert(title: 'T', content: '...'));

// UPDATE parcial (¡sin where afecta TODAS las filas!)
(update(todoItems)
  ..where((t) => t.title.like('%Important%'))
  .write(TodoItemsCompanion(category: Value(target.id))));

// UPDATE por primary key (reemplaza campos no-PK)
update(todoItems).replace(entry);

// UPDATE con expresión SQL (sin pasar por Dart)
update(users).write(UsersCompanion.custom(
  username: users.username.lower()));

// DELETE
(delete(todoItems)
  ..where((t) => t.id.isSmallerThanValue(10))).go();
```

Batch (inserciones masivas)

```
// atómico como transacción, optimizado para bloque
await batch((batch) {
  batch.insertAll(todoItems, [
    TodoItemsCompanion.insert(title: 'Uno', content: '..'),
    TodoItemsCompanion.insert(title: 'Dos', content: '..'),
    category: Value(3)),
  ]);
});
```

Upserts

```
// insertar o actualizar si ya existe (conflicto en PK)
into(users).insertOnConflictUpdate(user);

// upsert personalizado (p. ej. incrementar contador)
into(words).insert(
  WordsCompanion.insert(word: word),
  onConflict: DoUpdate((old) => WordsCompanion.custom(
    usages: old.usages + Constant(1))),
);
```

09 Transacciones

```
Future<void> deleteCategory(Category category) {
  return transaction(() async {
    await (update(todoItems)
      ..where((r) => r.category.equals(category.id)))
      .write(const TodoItemsCompanion(
        category: Value(null)));
    await delete(categories).delete(category);
  });
}
```

- await** todas las llamadas dentro del bloque; usar la transacción ya cerrada lanza excepción.
- Streams creados dentro se cierran al terminar.
- Anidadas (≥ 2.0): el rollback interno no revierte la externa si se captura la excepción.

07 Manager API (estilo ORM)

Generada por defecto; accesible vía `managers` en la BD.

```
managers.todoItems.get(); // todos
managers.todoItems.watch(); // stream reactivo
managers.todoItems.filter((f) => f.id(1)).getSingle();

// filtros combinables con & | y .not
managers.todoItems.filter((f) =>
  f.title("Title") | f.content.not.isNull());

// filtrar a través de relaciones (joins automáticos)
managers.todoItems.filter(
  (f) => f.category.description("School"));
managers.todoCategory.filter(
  (f) => f.todoItemsRefs((f) => f.id(1))); // back-ref

// prefetch de referencias (evita el problema N+1)
final rows = await managers.todoItems
  .withReferences((p) => p(category: true)).get();
for (final (todo, refs) in rows) {
  final cat = refs.category?.prefetchedData?.firstOrNull;
}

// escrituras
managers.todoItems.create(
  (o) => o(title: 'Nueva', content: '...'));
managers.todoItems.filter((f) => f.id(1))
  .update((o) => o(title: Value('Editada')));
managers.todoItems.filter((f) => f.id(1)).delete();

managers.todoItems.count(); // conteo
managers.todoItems.filter((f) => f.title("X")).exists();
```

10 DAOs (organizar el código)

Para no inflar la clase de BD, separa queries por dominio:

```
@DriftAccessor(tables: [TodoItems])
class TodosDao extends DatabaseAccessor<AppDatabase>
  with _$TodosDaoMixin {
  TodosDao(super.db);
  Future<List<TodoItem>> get allTodos =>
    select(todoItems).get();
}

// registrarlo en la base de datos:
@DriftDatabase(tables: [TodoItems], daos: [TodosDao])
class AppDatabase extends _$AppDatabase { ... }
// uso: database.todosDao.allTodos
```

11 Type converters y enums

```
// enums: intEnum guarda el índice; textEnum, el nombre
IntColumn get status => intEnum<Status>();

// converter propio (p. ej. objeto ↔ JSON)
class PreferencesConverter
  extends TypeConverter<Preferences, String> {
  Preferences fromSql(String s) =>
    Preferences.fromJson(json.decode(s));
  String toSql(Preferences v) => json.encode(v.toJson());
}

TextColumn get preferences => // uso en una columna
  text().map(const PreferencesConverter());
```

12 Migraciones

Cada cambio de esquema en una BD instalada requiere: **1)** subir `schemaVersion`, **2)** escribir la migración.

Manual (API básica)

```
int get schemaVersion => 3;

MigrationStrategy get migration => MigrationStrategy(
  onCreate: (m) async => m.createAll(),
  onUpgrade: (m, from, to) async {
    if (from < 2) await m.addColumn(todos, todos.dueDate);
    if (from < 3) await m.addColumn(todos, todos.priority);
  },
  beforeOpen: (details) async {
    await customStatement('PRAGMA foreign_keys = ON');
  },
);
```

Métodos del `Migrator`: `createAll()`, `createTable()`, `addColumn()`, `dropTable()`, `alterTable()` ...

Guiadas con `make-migrations` · **1.** configura `build.yaml`:

```
targets:
  $default:
    builders:
      drift_dev:
        options:
          databases: {my_database: lib/database.dart}
```

2. `dart run drift_dev make-migrations` (snapshot) · **3.** sube `schemaVersion` y repite: genera `database.steps.dart` y tests.

```
OnUpgrade get _schemaUpgrade => stepByStep(
  from1to2: (m, schema) async {
    await m.createTable(schema.groups);
  },
);
```

→ En desarrollo puedes borrar la `app/datos` y dejar que drift recree todo, sin escribir migraciones.

13 Tips rápidos

- `NativeDatabase.createInBackground(...)`: SQL en un isolate aparte, no bloquea la UI.
- `dateTime()`: timestamp Unix por defecto (o ISO-8601).
- `@DataClassName('Category')` renombra la data class.
- Tablas y queries **en SQL puro** con archivos `.drift` (verificadas en compile-time).
- Soporta web (WASM), escritorio, PostgreSQL y SQLCipher; incluye extensión para Flutter DevTools.

★ Flujo típico resumido

1. pubspec → drift, drift_flutter, drift_dev, build_runner
2. Definir tablas (`class X extends Table`)
3. `@DriftDatabase(tables: [...]) + schemaVersion`
4. `dart run build_runner build`
5. Consultas: `select/into/update/delete` o `managers.*`
6. UI reactiva: `.watch()` + `StreamBuilder`
7. Cambió el esquema → subir `schemaVersion` + migración